



分布与并行数据库

9 数据复制与一致性

主讲教师：鄂金龙
中国人民大学信息学院



主要内容

- 数据复制作用
- 数据复制一致性模型
- 分布式数据库复制策略
- 数据复制协议
- 大数据库一致性协议
- 大数据库复制一致性管理

数据复制作用

- 为满足可靠性和可用性，分布式数据库系统中的数据往往使用多个副本（拷贝），这些副本存储于不同的节点。
- 在分布式数据库中存储一个关系R，有以下几种方法：
 - 本地数据在本地数据库系统中存储
 - 系统维护关系R的几个完全相同的副本，各个副本存储在不同的节点上。与复制相对的方式是只存储关系R的一个副本。
 - 分片关系被划分为几个片段(垂直分片、水平分片或混合分片)，各个片段存储在不同的节点上，每个片段只有一个副本。
 - 分片关系被划分为几个片段，系统为每个片段维护几个副本，它们分别保存在不同节点上。

数据复制作用

- **数据复制**中，每个复制数据项 X 都有一系列副本 X_1 、 X_2 、...、 X_n ， X 称为逻辑数据项，并称它的副本为物理数据项。
- **复制透明性**：用户事务只需对逻辑数据项进行读写操作，副本控制协议自动将这些读写操作映射到物理数据项 X_1 、 X_2 、...、 X_n 上。系统逻辑上认为每个数据项只有一个副本。
- 数据不一致问题：在读写操作映射过程中，由于**复制协议**涉及到**更新的时机**、**内容**、系统的**体系结构**等，可能引起多个物理副本不一致。

数据复制作用

- 假设一个分布式数据库的两个副本存储于两个节点A和B，事务T1和T2开始之前数据项X的值均是100。



- 数据的不一致情况：
 - 情况1: $T1 < T2 \Rightarrow X = 160$
 - 情况2: $T2 < T1 \Rightarrow X = 180$
- 一致性和可用性相互矛盾：
 - 为了保证数据一致性，各个副本之间需要时刻保持强同步。
 - 当某一副本出现故障时，可能阻塞系统的正常写服务，影响系统的可用性
- 分布式数据库复制技术和一致性具有密切的关系，数据库设计时需要权衡系统的一致性和可用性。

数据复制一致性模型

- 从**客户端**的角度来看，**一致性**包括如下三种情况：
- **强一致性**：如客户A写入了数据项x的一个值到存储系统，存储系统保证客户A，B，C后续的读取操作都将返回x最新值。
 - **弱一致性**：设客户A写入了数据项x一个值到存储系统，存储系统不能保证A，B，C后续对x的读操作是否能够读取到最新值。
 - **最终一致性**：是弱一致性的一种特例。如A首先写入数据项x的一个值到存储系统，存储系统保证，如果后续没有新的写操作对x的值进行更新，则A，B，C的读取操作最终都会读取到A写入的值。从A写入数据x的值，到后续A，B，C读取到该值的这段时间，称为**不一致窗口**。不一致窗口的大小依赖于：交互延迟、系统的负载、同步的副本数。

数据复制一致性模型

□ **最终一致性**常见的变体有以下几种：

- **读自己写(read-your-writes)一致性**：如果用户A写入了数据项x的最新值，那么A的后续操作都会读到数据项x的该值。但是其他用户(B或C)可能要过一段时间才能看到数据项x的该值。
- **会话(session)一致性**：要求客户端和存储系统交互的整个会话期间保证读自己写一致性。如果原有会话因为某种原因失效而创建了新的会话，则存储系统不保证原有会话和新会话之间操作的读自己写一致性。
- **单调读(Monotonic read)一致性**：如果客户A已经读取了数据项x的某个值，那么A的后续操作将不会读取到数据x更早的值。
- **单调写(Monotonic write)一致性**：客户A的写操作按顺序完成，也就是说，对于同一个客户A对数据项x的操作，x在存储系统中的每个副本都必须按照与客户A相同的顺序完成。

数据复制一致性模型

- 从**存储系统**的角度看，一致性主要包含如下几方面：
 - **副本一致性**：存储系统中数据项 x 的多个副本之间的数据值是否一致、不一致的时间窗口等。
 - **更新顺序一致性**：存储系统中数据项 x 的多个副本之间是否按照相同的顺序执行更新操作。
- **存储系统**可以支持强一致性，也可以为了性能考虑仅支持最终一致性。
- 从**客户端**的角度看，一般要求存储系统能够支持读自己写一致性、会话一致性、单调读、单调写等特性。

分布式数据库复制策略

- 为了系统的可用性和可靠性，需要进行数据复制，需要考虑数据复制的时机、数据复制的内容和数据复制的体系结构等。
- 数据复制的执行方式
 - **同步复制**是指在事务进行更新时，将更新同时传播给其他所有副本，当某个数据项被事务更新后，该数据项的全部副本必须具有相同的值。
 - 同步复制能够保证应用程序的数据一致性，这样的复制协议称为**读一/写全协议**（read-one/write-all, ROWA）。
 - **不足：**
 - 由于事务使用2PC提交协议，更新的速度会受到系统中最慢的机器的限制。
 - 如果某一个副本损坏，将导致事务因无法完成在该副本上的更新而不能终止。
 - 系统需要各数据节点之间频繁通信以及时完成事务操作。

分布式数据库复制策略

- **异步复制**是指事务的提交不会等待更新作用到所有的副本中，而是当一个副本更新后，事务就会提交，其它副本在该更新事务提交之后的某个时间刷新。
- 异步复制中，首先更新数据的节点称为**主节点**，其他节点称为**从节点**。
 - **优点**：由于只需对一个副本提交更新即可，因而更新事务可以获得更短的**响应时间**；其次当目标系统崩溃时，该复制方法仍能够适应，只是复制工作将延迟到系统恢复后进行，具有**高可用性**。
 - **缺点**：由于副本的更新可以推迟到更新事务提交之后，导致副本之间的数据互相不一致，可能读到过时的数据；在有些情况下，可能产生“**事务倒转**”，即事务可能看不到自己已经写入的值。
- 异步复制不能满足强一致性，通常用于只要求弱一致性的应用中。

分布式数据库复制策略

- 数据复制的实现方法：数据对象复制、事务复制
- 数据对象复制
 - 把某一时刻源数据对象的内容通过网络复制到各节点的副本上，复制的内容是某一时刻的数据对象的状态也称为**快照**。
 - 数据对象复制**传输的是数据值**，是将整个发布内容复制给订阅者，可以是部分的行/列或者视图等。
 - 数据对象复制中往往需要复制较多的数据，对网络资源需求相对较高，不仅要求有较高的传输速度，而且要保证网络传输的可靠性

分布式数据库复制策略

□ 数据复制的实现方法：数据对象复制、事务复制

□ 事务复制

- 把修改源数据库的**事务操作**发送到副本节点，可以是修改的表项、事务或事务日志。
- 副本接收到复制内容后，通过在**本地数据库**执行接收到的事务操作来实现与源数据或者处理过程的一致性。
- 事务复制在网络中传送的是事务，即将发生的变化传送给订阅者，是一种**增量复制**。
- 在事务复制中，要不断监视源数据库的数据变化，因而**主服务器的负担较重**。当发布数据发生变化时，这种变化很快会传递给订阅者，而不像数据对象复制那样等待一个相对较长的时间间隔。

分布式数据库复制策略

□ 数据复制体系结构：主从式、对等式、级联式

□ 主从式复制

- 最基本的一种复制结构，每个数据项定义一个**主副本**，更新首先在主副本中进行，然后再传播到其他副本中（称为**从副本**），存放主副本的节点称为主节点，存放从副本的节点称之为**从节点**。如果所有复制数据只有一个主节点，则称为**单主节点复制**。
- **优点：**数据更新过程比较**简单**，更新只需在主节点上完成，不必与多个从节点同步，可以保证每个数据项至少其主节点的副本中存放着最新的值。
- **缺点：**对于单主节点的结构，所有数据的主副本都存于一个节点中，那么这个节点将**负载过重**，成为系统瓶颈，导致系统性能下降。
- **应用场景：**这种复制技术适合于**数据仓库**和集中在一个或少数几个节点上的应用数据的情况。

分布式数据库复制策略

□ 数据复制体系结构：主从式、对等式、级联式

□ 对等式复制

- 首先更新事务所在节点的**局部副本**，然后再**传播给其他副本节点**，不同事务可以更新位于不同节点上的同一数据项的副本，所有副本在任何节点都可以被修改，并且修改可以发送给其他副本，即所有节点的地位、作用是等同的，没有主从关系。
- **优点：**可以避免单一节点或者少数节点任务过载，产生系统瓶颈的情况，提高系统的吞吐量。
- **不足：**相对主从式复制技术，对等复制技术**比较复杂**。一个数据项的不同副本可能在不同节点上同时更新，需要考虑**并发控制**问题，并需要结合同步复制技术和异步复制技术实现数据一致性
- **应用场景：**对等复制适于拥有分布式决策和操作中心的**协作应用**，可使系统各节点负载均衡，与异步复制技术结合进一步提高系统的可用性。

分布式数据库复制策略

- 数据复制体系结构：主从式、对等式、级联式
- **级联复制**
 - 级联复制技术是主从结构的一个扩展，它也是由一个主副本和若干个从副本组成。
 - 不同于主从结构的是，它允许每个从副本（从属节点）都具有复制的能力，即一个从副本可以把接收到的复制数据再传给下一个从副本。

数据复制协议

□ 常用的主从复制协议和对等式复制协议

□ 主从复制协议

两个阶段：集中式事务接收/拒绝阶段和集中式事务执行阶段。

■ 阶段1：集中式事务接收/拒绝阶段

- 步骤1：事务到达一个节点，本地事务监视器（TM）向该节点的从节点发送一个请求要求运行该事务；
- 步骤2：从节点向主节点发送运行该事务的请求；
- 步骤3：主节点接到从节点的请求，则检测活动事务之间的冲突。主节点维护一个保存所有被主节点认可的活动的活动事务的一个暂时挂起队列，如果新请求与暂时挂起队列中的活动事务不存在任何冲突，主节点将该事务加入暂时挂起队列中，并向从节点发送“ACK+”，否则发送“ACK-”；
- 步骤4：从节点如果收到“ACK+”的响应，则将进入事务运行阶段，如果收到“ACK-”响应，则拒绝执行该事务。

数据复制协议

□ 常用的主从复制协议和对等式复制协议

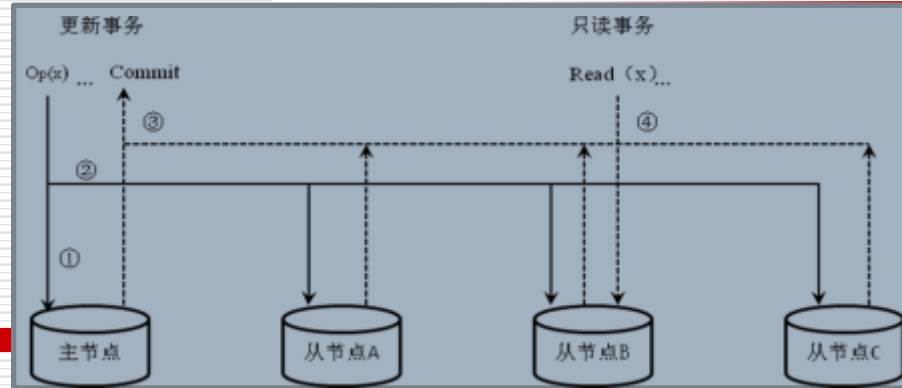
□ 主从复制协议

两个阶段：集中式事务接收/拒绝阶段和集中式事务执行阶段。

■ 阶段2：集中式事务处理阶段

- 步骤1：从节点收到来自主节点的“ACK+”，则向其他所有的从节点发送一个消息广播该更新请求；
- 步骤2：收到该请求的从节点必须运行该事务并且向发送该请求的从节点发送一个ACK消息确认已收到请求；
- 步骤3：发送请求的从节点等待得到来自其他从节点的通知，则事务在所有节点的数据库副本已经被更新成功；
- 步骤4：从节点给主节点和TM发送一个DONE消息，确认事务成功；
- 步骤5：主节点收到DONE消息后，将该事务从暂时挂起队列中移除；

数据复制协议



- 常用的主从复制协议和对等式复制协议
- 主从复制协议-**结合同步更新的复制协议**：结合同步复制，依据已知主副本位置（有限复制透明）和由TM负责确认主副本位置（完全复制透明）两种不同的数据复制策略分别制定复制协议。
 - **有限复制透明性**：假设数据复制体系结构为单主节点。并且应用知道主节点位置，使用有限复制透明技术。
 - **步骤1**：主副本中，如果操作是 $\text{Read}(X)$ ， X_M 获得读锁，读取数据，返给用户。如果操作是 $\text{Write}(X)$ ，则 X_M 获得写锁，在主副本上执行写操作；
 - **步骤2**：主TM发送Write操作到每个从节点，保证冲突更新在每个从节点上的执行顺序和在主节点上的执行顺序保持一致。
 - 对于从节点提交的只读事务 $\text{Read}(X)$ (不包含写操作)，将发送给主节点，然后获得读锁，在主节点使用集中式封锁协议，读操作可以在主节点上执行，并将结果返回给应用程序，也可以发送“lock granted”消息给对应的从节点，使从节点局部执行读操作。

数据复制协议

- 常用的主从复制协议和对等式复制协议
- 主从复制协议-**结合同步更新的复制协议**：结合同步复制，针对应用已知主副本位置（**有限复制透明**），和由TM负责确认主副本位置（**完全复制透明**）两种不同的数据复制策略分别制定复制协议。
 - **完全复制透明性**
 - **有限复制透明协议**中，要求应用**知道单主节点的位置**，并且主节点执行需要处理读写操作，在2PC执行过程中充当协调者的工作，负载过大。
 - **完全复制透明协议**：为了解决这些问题，使用事务管理程序(TM)，更新事务不是提交给主节点，而是提交给执行应用节点上的TM，TM可以起到更新和只读事务的协调者的作用，应用程序可以简单地提交事务给各自的局部TM，从而替代了主节点的部分工作，复制也成为完全透明的。完全复制透明协议的实现步骤如下：

数据复制协议

- 常用的主从复制协议和对等式复制协议。
- 主从复制协议-**结合同步更新的复制协议**：结合同步复制，针对应用已知主副本位置（**有限复制透明**），由TM负责确认主副本位置（**完全复制透明**）两种不同的数据复制策略分别制定复制协议。

■ 完全复制透明性

步骤1：协调TM接受操作，并发送到主节点；

步骤2：如果操作是Read(x)，则给 X_m 加读锁，并通知TM。然后，TM可以将Read(X)转发给任意一个保存着X副本的从节点，读取操作可以在相应从节点上由数据处理程序完成。

步骤3：如果操作是Write(x)，那么主节点执行下列操作：

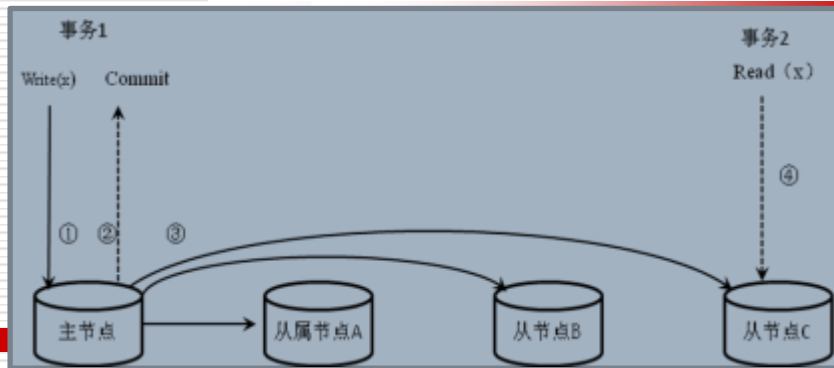
步骤3.1：给X加写锁；

步骤3.2：调用局部数据处理程序，并在自己所属的X的副本上执行Write(X)；

步骤3.3：通知TM，它得到了所需的写锁。

步骤4：TM将Write(x)发送给所有保存X的副本的从节点，然后每个从节点的数据处理程序在局部副本上执行相应的Write操作。

数据复制协议



- 常用的主从复制协议和对等式复制协议。
- 主从复制协议-结合异步更新的复制：结合异步更新，更新操作仍然首先会作用在主副本中，然后再传播到其他从节点上。

■ 有限复制透明性

主副本接收到请求时，执行以下步骤：

- 步骤1：主节点中对于写操作，包含写操作的事务首先在主副本中执行；
- 步骤2：事务在主节点中提交；
- 步骤3：刷新事务发送给从节点

从节点接收到操作请求时，执行以下步骤：

- 步骤1：如果操作是Read(X)，则读取局部副本，并将结果发送给用户
- 步骤2：如果从节点接收到的Write(X)均要被拒绝，取消相应的事务；
- 步骤3：从节点接收到刷新事务请求时，则更新自己的局部副本；
- 步骤4：当接收到Commit或者Abort命令时，则局部执行对应的操作；

数据复制协议

- 常用的主从复制协议和对等式复制协议。
- 主从复制协议-**结合异步更新的复制**：结合异步更新，更新操作仍然首先会作用在主副本中，然后再传播到其他从节点上。
 - **完全复制透明性**
 - 包含Write(X)的事务在提交时，主节点给事务加上时间戳，并将主副本的时间戳设置为最后一个修改过它的事务的时间戳，当从节点接收到刷新事务时，将副本的值统一设置为主副本对应事务的最后时间戳。
 - 事务在主节点上的时间戳应该大于所有已有的时间戳，并且小于它所访问数据项的时间戳，如果这个时间戳无法生成，就取消事务。
 - 这种方法可以保证读操作可以读到正确的值。对于事务不能看到自己写操作的结果，可以维护一个所有更新操作的列表，当有Read执行时，查询该列表。这个列表由主节点维护，读写操作都在主节点上完成。

数据复制协议

- 常用的主从复制协议和对等式复制协议。
- 对等式复制协议-**结合同步复制的算法**：在对等的复制结构中，所有的节点没有主从之分，结合同步更新，该算法满足强一致性。该算法分为两个阶段：
 - **阶段1：分布式事务接收阶段**
 - **步骤1**：事务进入一个给定节点，本地TM向存储数据项的其他从节点发送一个本地请求，要求运行该事务。
 - **步骤2**：从节点接收请求后向其他从节点发出请求询问是否可以运行该事务。
 - **步骤3**：发出请求的从节点等待来自其他从节点的反馈。
 - **步骤4**：如果所有从节点都同意运行该事务，则发出请求的从节点进入事务执行阶段，否则，如果存在不同意运行该事务的节点，那么发出请求的节点拒绝执行该事务。

数据复制协议

- 常用的主从复制协议和对等式复制协议。
- 对等式复制协议-同步复制结合的复制算法：

针对每个从节点接收来自不同其他从节点的操作请求，可能包含对同一数据项的不同操作（可能存在冲突操作），制定**基于优先权的投票**规则。主要包括以下几种情况（假定节点支持事务 T_i ）：

- **情况1：**参与投票的从节点目前没有运行事务，则该从节点对执行事务 T_i 投票OK。
- **情况2：**投票的从节点上正运行着事务 T_j ，但 T_j 与 T_i 之间没有冲突，则该从节点对执行事务 T_i 投票OK。
- **情况3：**投票的从节点上正运行着事务 T_j ，且 T_j 与 T_i 之间存在冲突。为了解决冲突，事务的时间戳可以作为优先考虑的因素。由于 T_j 先于 T_i 启动，并正在运行，则比 T_i 有更高的优先权，因而，该从节点对 T_i 投票NOT OK，并撤销 T_i 。反之，如果 T_i 比 T_j 先启动，具有较高优先权，则该从节点对 T_i 投票OK，并撤销 T_j 。

数据复制协议

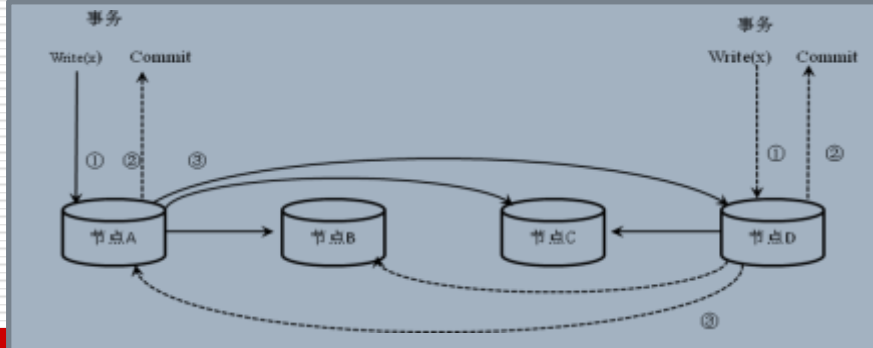
- 常用的主从复制协议和对等式复制协议。
- 对等式复制协议-**同步复制结合的复制算法**:

- **阶段2：分布式事务执行阶段**

从节点收到其他所有从节点的OK消息之后，开始事务的执行阶段，具体步骤：

- **步骤1**：从节点发送新广播让所有副本执行事务。
- **步骤2**：收到该请求的从节点执行事务后，发送“ACK”回答发出请求的从节点。
- **步骤3**：发出请求的从节点等待所有其他从节点的回应，如果获得全部回应，则认为事务已经在所有节点上执行成功。
- **步骤4**：从节点通知TM事务成功执行。

数据复制协议



□ 常用的主从复制协议和对等式复制协议。

□ 对等式复制协议-与**异步复制结合的复制算法**:

在对等结构的复制中，更新可以在任何副本上发生，结合异步更新算法，则可以延迟地传播给其它副本。协议步骤如下：

- **步骤1**：事务在本地节点上提交操作请求，每个Read和Write操作都作用于局部副本上，并且事务局部提交，提交之后，更新结果成为永久的。
- **步骤2**：事务提交之后，更新信息由刷新事务传播给其他节点。
- **步骤3**：刷新事务到达其他节点后，参加事务的局部调度。可能产生多个事务更新同一数据项的请求（可能是相互冲突的），按照冲突调度算法，依次更新数据项

具体的冲突调度算法：可以按照时间戳的顺序制定，比如最新时间戳的事务胜出，也可以设置调度次序使其倾向于某些发生更新的源节点，当这些节点更重要时。

大数据库一致性协议

□ **一致性协议**是一致性模型的实现方法的描述。本部分介绍在大数据库中使用的几种典型的一致性协议和技术：
Paxos协议、反熵协议、时钟向量技术。

□ **Paxos协议**

- **Paxos**用于实现分布式系统多个节点数据一致性的协议。通过消息传递的方式，在多个冲突请求中选出一个从而达到分布式系统中各节点的数据一致。
- **冲突产生的原因**：可能是由于主节点失效、其他多个节点自我选举成为新的主节点，也可能由于节点上的数据修改操作引起的。

大数据库一致性协议

□ Paxos算法划分**3个角色**，如下

- **proposer**（提出者）：提出者可以提出提议以供投票。
- **acceptor**（批准者）：批准者可以对提出者提出的提议进行投票表决，从众多提议中选出唯一确定的一个。
- **learner**（学习者）：学习者无对提议的投票权，但是可以从批准者那里获知是哪个提议最终被选中。
- 为保证一致性，**Paxos**协议规定：提议只有被提出后才能被选择，算法的一次执行只能选择一个提议，提议只有被选中之后才能通知其他节点。
- 实现中，允许一个进程（**process**）扮演多个代理，代理之间用消息通信。通信采用异步、非拜占庭模型（**non Byzantine model**），即允许消息的丢失或者重复，但是不会出现内容损坏的情况。

大数据库一致性协议

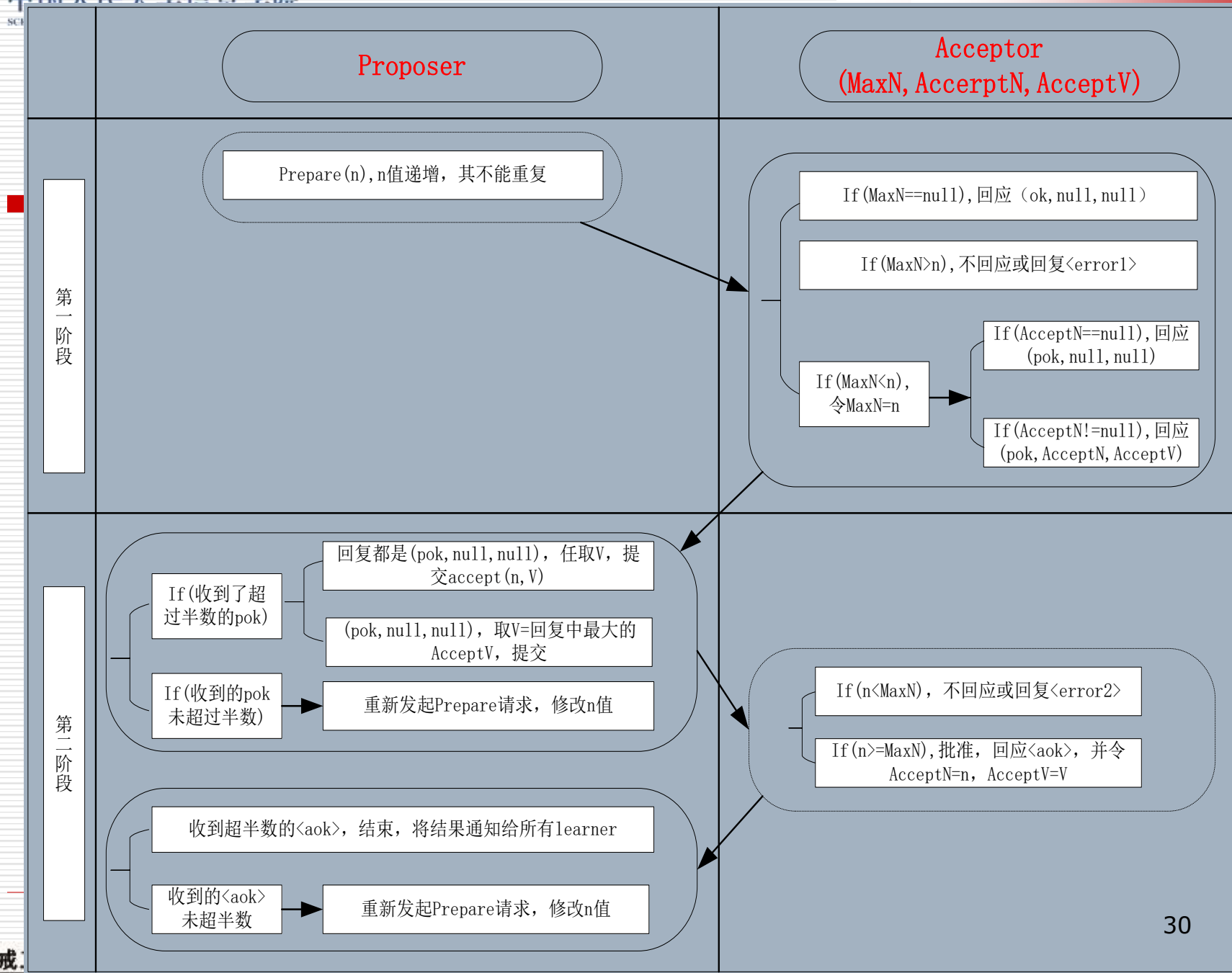
□ Paxos算法包括2个阶段

■ 阶段1：准备（Prepare）

1. 一个proposer选择一个提议编号n，并将一个编号为n的prepare请求发送给大多数acceptor。
2. 如果Acceptor接收到的prepare请求的编号n大于它已经回应的任何prepare请求，它就回应已经批准的编号最高的提议（如果有的话），并承诺不再回应任何编号小于n的提议。

■ 阶段2：批准（accept）

1. 提议如果得到了来自超过半数的Acceptor对prepare请求的回应（编号为n），那么就向Acceptor发送提议编号为n，值为v的accept请求。其中，n是阶段1中发送的序号，v是收到的所有回应中编号最大的提议的值，若收到的回应全部是null，那么v自定义。
2. 如果Acceptor收到了一个编号为n的accept请求，就批准这个提议，除非它已经给某个prepare请求回应了编号大于n的提议。



大数据库一致性协议

□ Paxos算法-总结

- 某一时刻，某个提议被多数Acceptor批准(Accept)后，被批准的Proposal Value一定和这个Proposal Value相同。
- Paxos算法是个民主选举的算法——大多数的决定会成为整个集群的统一决定。
- 任何一个节点都可以提出要修改某个数据的提议，是否通过这个提议取决于这个群中是否有超过半数的节点同意。

大数据数据库一致性协议

□ 反熵协议

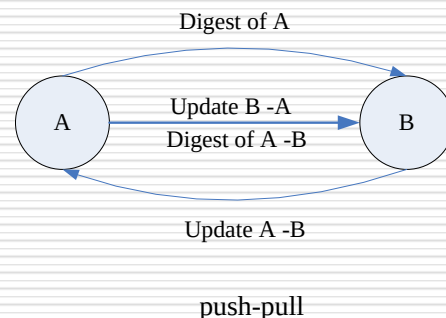
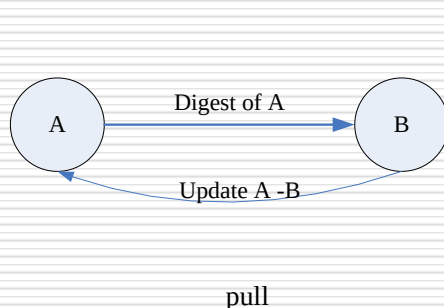
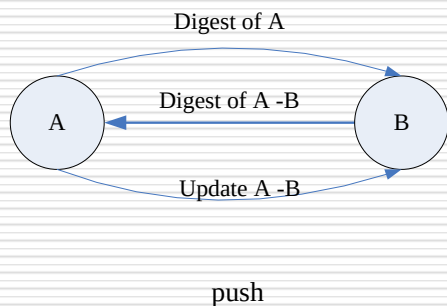
- 反熵（Anti-Entropy）是Gossip算法中的一种，Gossip是一种对复制状态不要求强一致性的有效方法。更新传播在预期的时间内按参与节点数目的对数增长，没有什么限制条件，即使某些节点失效或消息丢失，经过一段时间后，这些节点的状态也会与其他节点达成一致，即系统可以获得最终一致性。
- Gossip算法包括反熵和谣言传播（rumor mongering）
 - 反熵协议中，消息被传递一直到它被新的消息所替代。反熵协议可用于在一组参与者中共享可靠消息。
 - 谣言传播协议中，参与者传播持有信息足够长的时间，以便于所有的参与者接受到这个消息。

大数据库一致性协议

□ 反熵协议

□ 系统中的两个节点（A、B）之间存在三种通信方式：

- **push**: A节点将数据推送给B节点，B节点更新A中比自己新的数据。
- **pull**: A将摘要数据 (node, key, value, version) 推送给B，B根据摘要数据来选择那些版本号比A高的数据推送给A，A更新本地。
- **push-pull**: 与pull类似，只是多了一步，A再将本地比B新的数据推送给B，B更新本地。



在真实应用中将全部数据都推送出去需要消耗太多代价！

大数据库一致性协议

□ 反熵协议

- 为了**保证一致性**，节点p中的数据由(key, value, version)构成，规定数据的值(value)及版本号(version)只有宿主节点才能修改，其他节点只能间接通过协议来请求数据对应的宿主节点修改，即消息m(p)只能由节点p来修改。反熵协议通过**版本号大小**来对数据进行更新。
- **协调机制**可以使用**精确协调**和**整体协调**两种
 - **精确协调**：节点相互发送对方需要更新的数据，每个数据项独立地维护自己的version，每次交互是把所有的(key,value,version)发送到目标进行比对，从而找出双方不同之处从而更新。由于消息存在大小限制，因而需要选择一些数据进行发送，可以随机选择，也可确定性的选择数据，比如最老版本优先或者最新版本优先的原则。
 - **整体协调**：为每个节点上的宿主数据维护统一的version，相当于把所有的宿主数据看作一个整体，当与其他节点进行比较时，只需这些宿主数据的最高version，如果最高version相同，说明这部分数据全部一致，否则再进行精确协调。

大数据数据库一致性协议

□ NWR协议

为保证数据的不同副本的一致性，很多系统采用了NWR模型，模型中各符号含义如下：

- **N**：复制的节点数量
- **R**：成功读操作的最小节点数
- **W**：成功写操作的最小节点数

在包含N个副本的系统中，要求写入至少W份副本，至少读取R个副本才认为成功。

- 如果要求系统**满足强一致性**，配置要求 **$W+R>N$** ，因为 $R>N-W$ ，也就是读取的份数一定要比总副本数减去确保写成功数的差值要大。满足上述条件使得客户端每次读取，都**至少读取到一个最新的版本**，从而不会读到一份旧数据。

大数据数据库一致性协议

□ NWR协议

- 如果要**高的一致性**：可以配置 $N=W$ ， $R=1$ ，但可用性就会大大降低。如果要**高的可用性**：可以配置 $W=1$ ，使得写操作延迟最低，同时通过异步的机制更新剩余的 $N-W$ 个节点。
- **总结**
 - $W=1$ ， $R=N$ ，对写操作要求高性能和高可用性。
 - $R=1$ ， $W=N$ ，对读操作要求高性能和高可用性，比如类似cache之类业务。
 - $W=Q$ ， $R=Q$ ，其中 $Q=N/2+1$ ，一般应用适用，读写性能之间取得平衡，如 $N=3$ ， $W=2$ ， $R=2$ 。
 - NWR模型把CAP的选择权交给了用户，让用户自己选择CAP中的哪两个。



大数据库一致性

□ 向量时钟技术

- **向量时钟(Vector Clock)**是一种在分布式环境中为各种操作或事件产生偏序值的技术，最早应用于分布式操作系统的进程间事件同步。
- 在**分布式数据库**中，使用**向量时钟**来维护各不同节点的事件时间戳，从而来检测操作或事件的并行冲突，用来**保持系统的一致性**。
- 在**分布式环境**中，**向量时钟**描述来自不同**节点的时钟值** $V_i[1]$ 、 $V_i[2]$ 、...、 $V_i[n]$ 构成的向量，其中 $V_i[j]$ 表示第*i*个节点维护的第*j*个节点上的时钟值。
- **向量时钟的取值**可以是来自节点本地时间的时间戳或者是根据某一规则生成的有序数字。
- 在分布式数据库中 $V_i[j]$ 的**值代表了数据的版本信息**，也就是说 **$V_i[n]$ 是从节点*i*上维护的节点*n*上的数据版本信息**。通过向量时钟，每个节点可以知道其他节点或副本的状态。

例如：假设系统中包含3个副本，其中包括时钟向量
 $V_0(3,2,0)$ ， $V_1(1,3,0)$ ， $V_2(0,0,1)$ 。

- 节点0上读到的节点0自身的时钟值为“3”，节点1的时钟值为“2”，节点2的时钟值为“0”；
- 节点1上读到的节点0的时钟值为“1”，节点1自身的时钟值为“3”，节点2的时钟值为“0”；
- 节点3上读到的节点0的时钟值为“0”，节点1时钟值为“0”，节点2的自身的时钟值为“1”

大数据库一致性协议

□ 向量时钟技术

□ 向量时钟的更新规则

- **规则1**：每个节点的初值置为0。每当有数据更新发生，该节点所维护的时钟值将增长一定的步数 d ， d 的值通常由系统提前设置好。该规则表明，如果操作 a 在操作 b 之前完成，那么 b 的向量时钟值大于 a 向量时钟值。
- **规则2**：在节点 i 的数据更新之前，我们对节点 i 所维护的向量 V_i 进行更新： $V_i[i] = V_i[i] + d$ ($d > 0$)。该规则表明，当 $V_i[i]$ 处理事件时，其所维护的向量时钟对应的自身数据版本的时钟值将进行更新。
- **规则3**：当节点 i 向节点 j 发送更新消息时，将携带自身所了解的其他节点的向量时钟信息。节点 j 将根据接收到的向量与自身所了解的向量时钟信息进行比对，然后进行更新： $V_j[k] = \max\{V_i[k], V_j[k]\}$ 。在合并时，节点 j 的向量时钟每一维的值取节点 i 与节点 j 向量时钟该维度值的较大者。

大数据数据库一致性协议

□ 向量时钟技术

■ 优势

- 节点之间不需要同步时钟，即不需要全局时钟，但是需要在所有节点上存储、维护一段数据的版本数。
- 比较简单的冲突解决方案：
 - 随机选择一个数据的版本返回给用户。
 - 在Dynamo中系统将数据的不一致性冲突交给客户端来解决。当用户查询某一数据的最新版本时，若发生数据冲突，系统将把所有版本的数据返回给客户端，交由客户端进行处理。

■ 缺点

- **向量时钟值的大小**与参与的用户数量有关。若参与的用户很多，随着时间推移，向量时钟值会很大。一些系统中为向量时钟记录**时间戳**，某一时刻根据记录的时间对向量时钟值进行裁剪，删除最早记录的字段。
- 向量时钟在实现中需要考虑两个**主要问题**：如何确定持有向量时钟值的用户，如何防止向量时钟值随着时间不断增长。

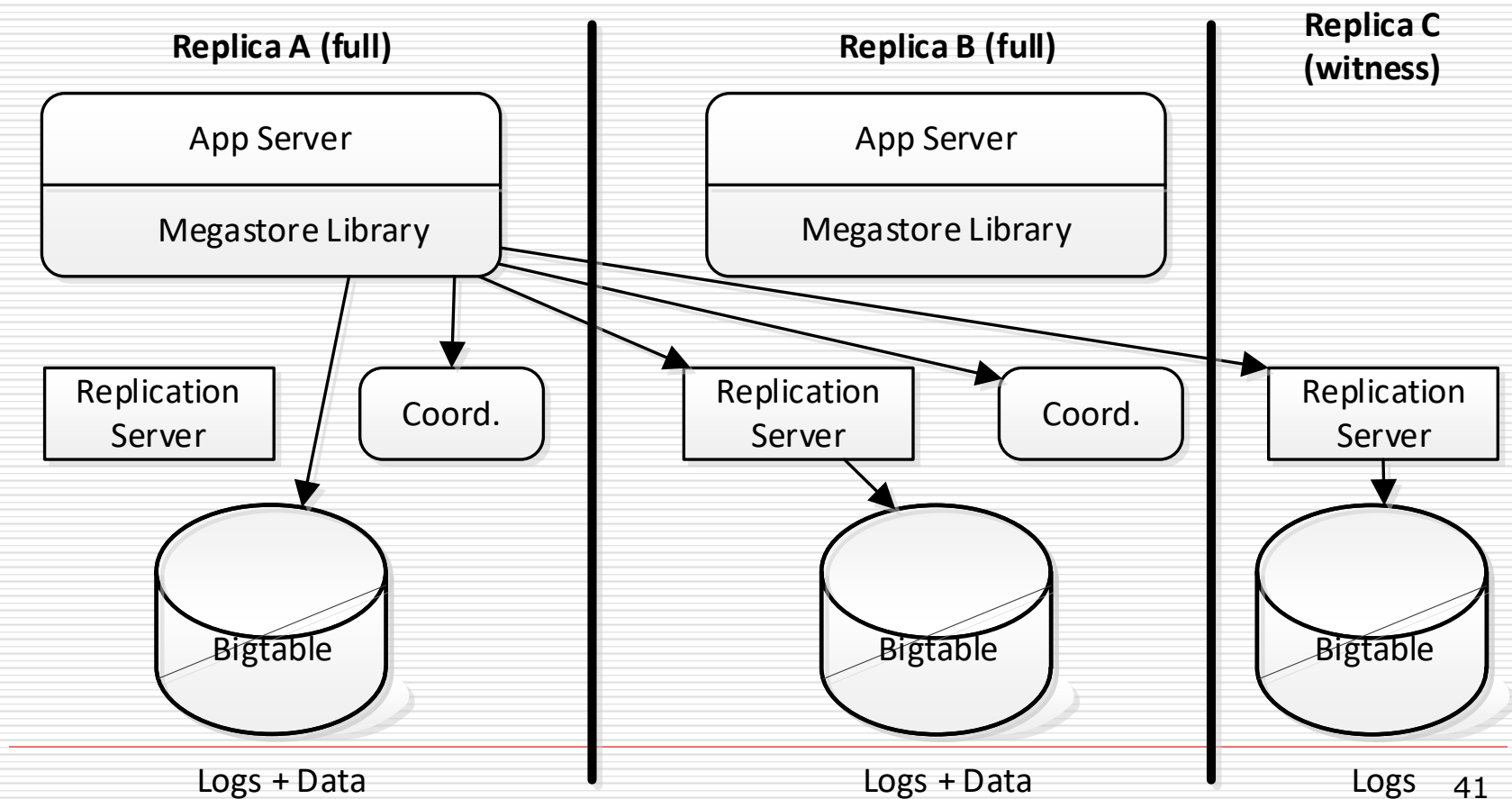
大数据数据库复制一致性管理

□ 基于Paxos的复制管理技术

- Google产品广泛使用了Paxos协议，Megastore、Chubby和Spanner等一系列产品都是在Paxos协议的基础上实现一致性的。
- Megastore使用了同步复制技术，其复制系统向外提供了一个单一的、一致的数据视图，读和写能够从任何副本开始，并且无论从哪个副本的客户端开始，都能保证事务的ACID特性。

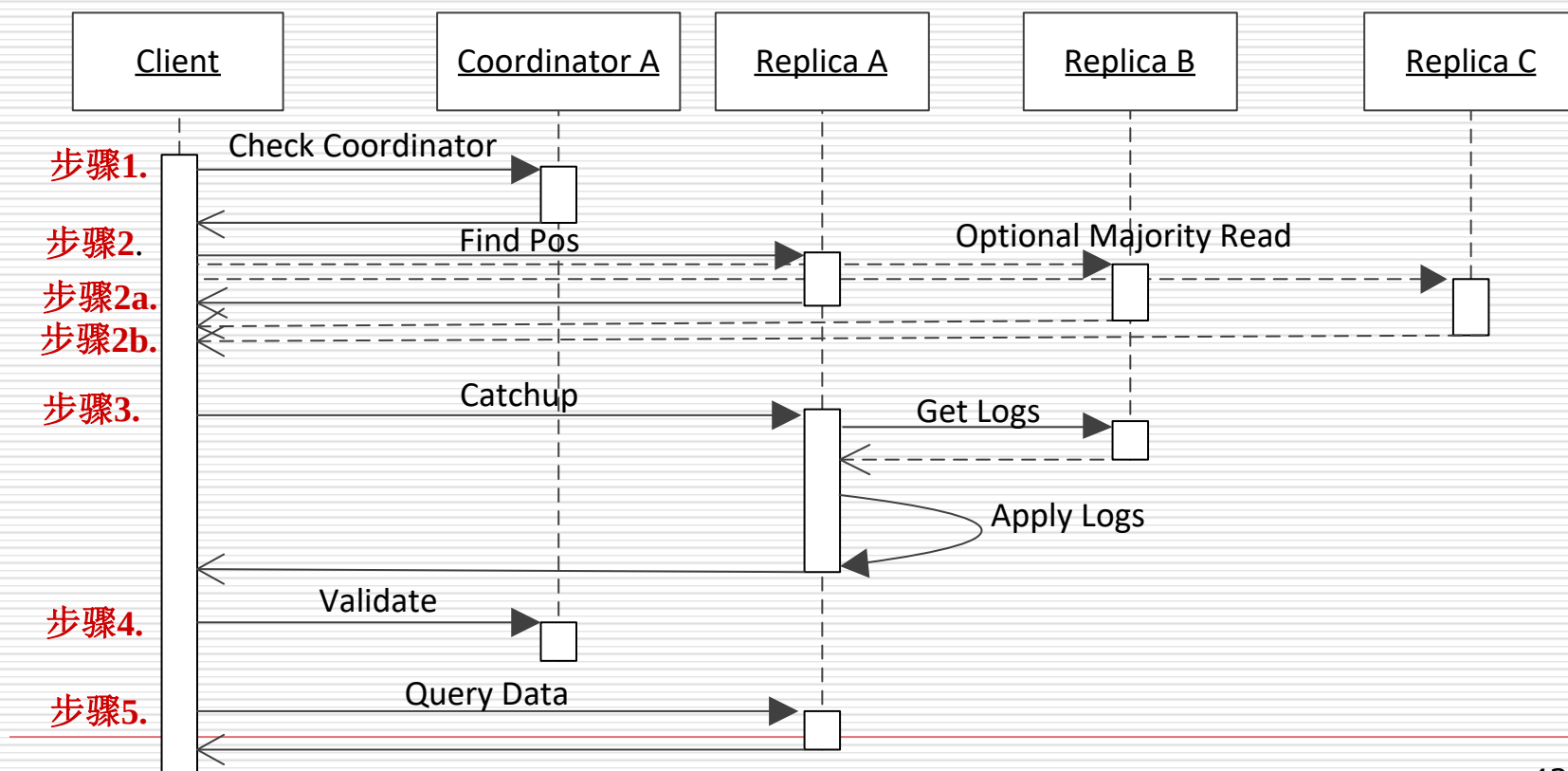
大数据数据库复制一致性管理

□ 基于Paxos的复制管理技术：Megastore架构实例



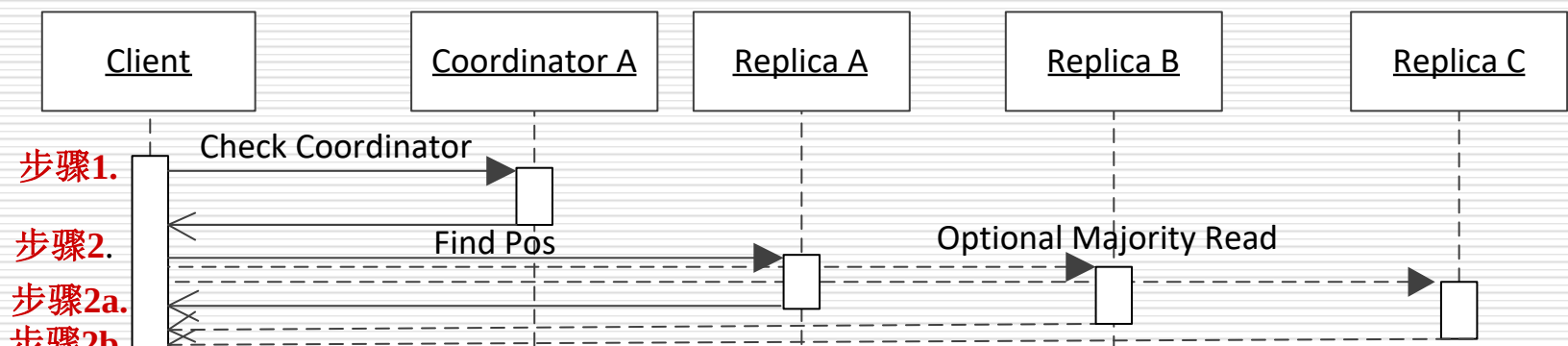
大数据数据库复制一致性管理

□ 基于Paxos的复制管理技术-最新读



大数据数据库复制一致性管理

□ 基于Paxos的复制管理技术-最新读



步骤1. 本地查询：查询本地副本的协调者，判定当前副本的Entity Group是否为最新。

步骤2. 查找位置：确定最高的可能已提交的日志位置，然后选择一个应用这个日志位置生效的副本。

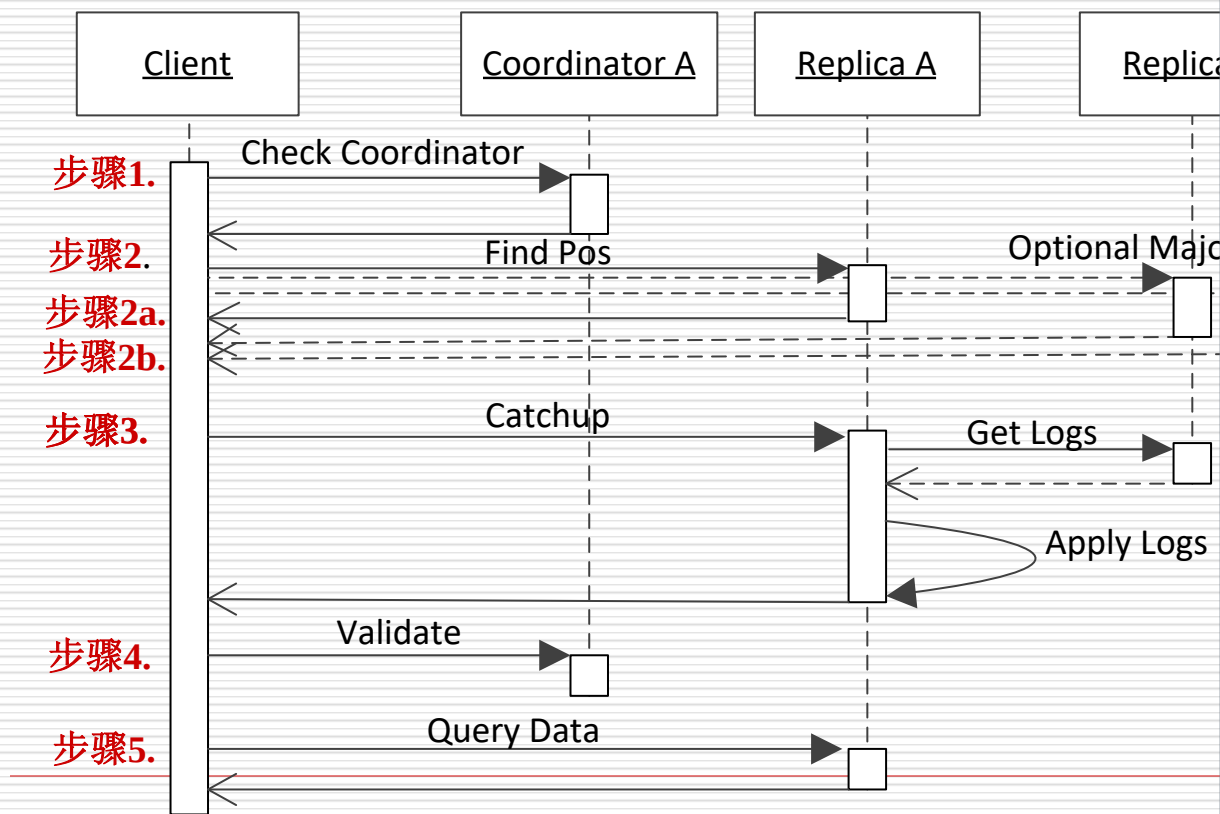
步骤2a. 本地读：如果步骤1发现本地副本是最新的，从本地副本中读取最高的被接受的日志位置和时间戳。

步骤2b. 多数读：如果本地副本不是最新的(或者步骤1或步骤2a超时)，那么从一个多数派副本中发现最大的日志位置，然后选取一个读取。从中选取一个最可靠的或者最新的副本，不一定总是本地副本。



大数据数据库复制一致性管理

□ 基于Paxos的复制管理技术-最新



步骤3.追赶：当一个副本选中之后，按照下面的步骤追赶到已知的日志位置：

步骤3a.对于被选中的不知道一致值的副本中的每一个日志位置，从另外一个副本中读取值。对于任何一个没有已知已提交的值的日志位置，通过Paxos发起一个无操作的写操作。Paxos将会驱动多数副本的一个值获得一致——可能是无操作的写操作或者是之前提议的写操作。

步骤3b.顺序地在所有没有生效的日志位置生效成一致的值，并将副本的状态变为到分布式一致状态。

如果步骤3的追赶过程失败，在另外一个副本上重试。



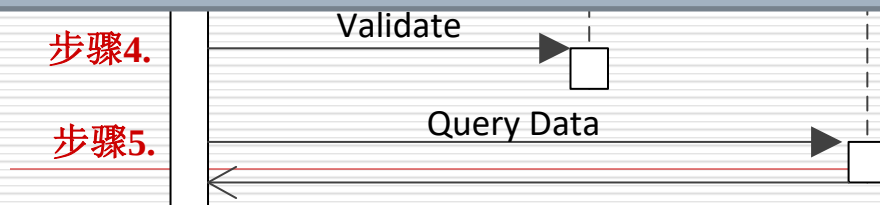
大数据数据库复制一致性管理

□ 基于Paxos的复制管理技术-最新读



步骤4. 验证: 如果本地副本被选中并且之前没有最新的副本, 发送一个验证消息到协调器, 以判断(entity group, replica)元组能够反馈所有已提交的写操作。这里无需等待回应, 因为如果请求失败, 下一个读操作会重试

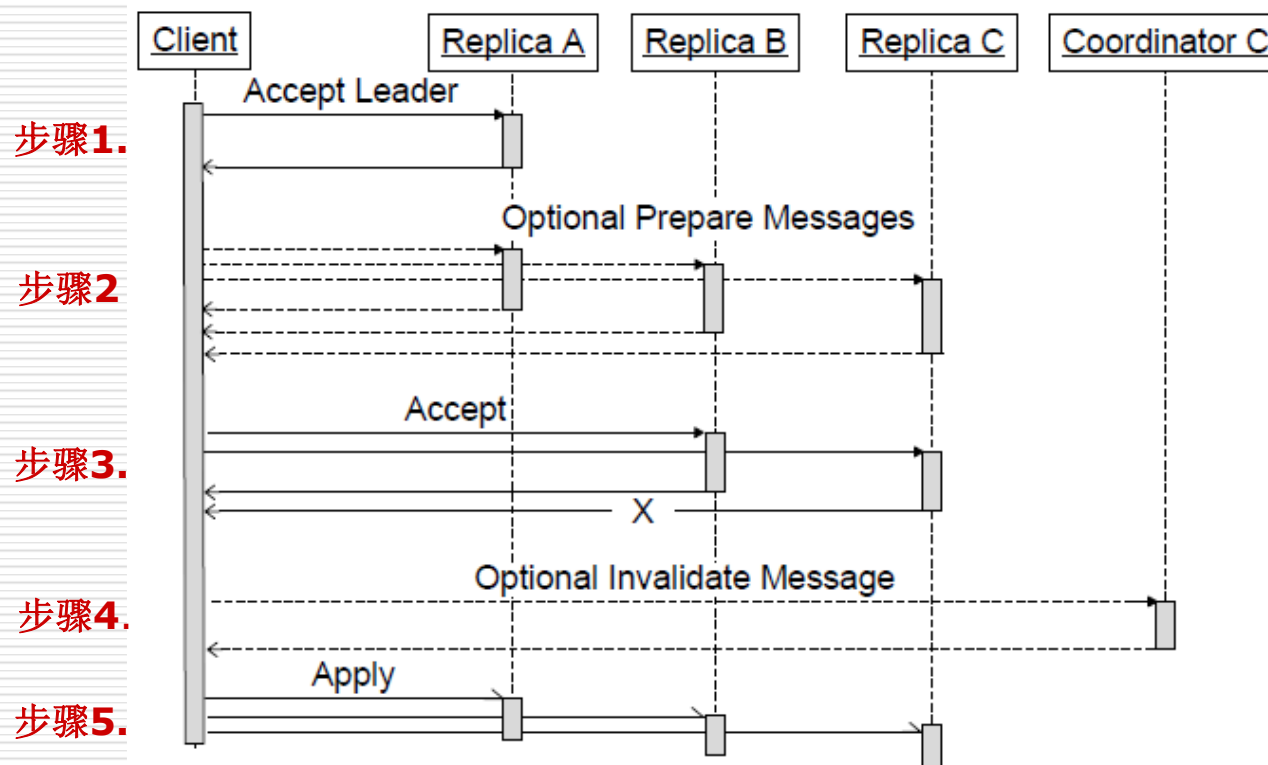
步骤5. 查询数据: 使用被选择的日志位置的时间戳, 从选中的副本中读取数据。如果选中的副本不可用, 选取另外一个副本重新开始执行追赶, 然后从它那里读取。一个大的读取结果有可能从多个副本中透明地读取并且组装返回。





大数据数据库复制一致性管理

□ 基于Paxos的复制管理技术-写操



步骤1. 接受Leader:

请求Leader接受值做为0号提议的值。如果成功。跳到步骤3;

步骤2. 准备:

在所有副本上执行 Paxos Prepare阶段，使用一个比当前任何日志位置都更高的提议号，将值替换成拥有最高提议号的值;

步骤3. 接受:

请求余下的副本接受这个值。如果多数副本失败，转到步骤2;

步骤4. 失效:

将没有接受值的完全副本的协调器置为失效。

步骤5. 生效:

将值的更新在尽可能多的副本上生效。如果选择的值与原始提议的值不同，则返回冲突错误。46





第九章 结束

